Princess Sumaya University for Technology
جامعة الأميرة سميّة للتكنولوجيا

Computer Science Department

Second Exam with Answer Key

CS-11313: Algorithm Analysis and Design

Fall Semester 2022/2023 Thursday, 15-Dec-22@ 3:00 PM

Student Name:		**Student Number:**
Instructor:	**Section # (or time):**	**Signature:**

Instructions

- Time allowed: **60 Minutes**
- This booklet should be 7 pages long, including this page. If not, report this to the instructor.
- Answer all questions in the examination paper provided.
- Write your answers clearly: you may not receive full credit if your answer is unreadable.

Question	Max Mark	Your Mark	CLO
1	**30**		CLO 1
2	**21**		CLO 2
3	**20**		CLO 2
4	**29**		Clo3
Total	**100**		

Question 1: Recurrences [30 Points]

(a) Solve the following three recurrences using the **Master Theorem** if possible. Show ***all*** steps for full credit. If it is not possible to solve the problem using the Master Theorem, state so and explain why.
(18 points)

1. $T(n) = 9T(n/9) + 3n^2$

2. $T(n) = 27T(n/3) + 3\,n^2 \log n$

3. $T(n) = 4T(n/2) + n^2/\log n$

(b) Solve the following recurrence using the **Iterative substitution method**. (12 points)

$T(n) = n + 2T(n/2)$, where $T(1)=1$

Second Exam Answer key
Question 1a .1
$T(n) = 9T(n/9) + 3n^2$

$Number\ of\ leaves = n^{log_b a} = n^{log_9 9} = n$
$f(n) = 3n^2$

Case 3:
$f(n) = \Omega(n^{log_b a + \epsilon})$
$3n^2 = \Omega(n^{1+\epsilon})\ \ for\ \epsilon > 0, choose\ \epsilon = 0.5\ \Rightarrow\ 3n^2 = \Omega(n^{1.5})\ is\ true$

Next, Check the regularity condition $af(n/b) \leq kf(n)\ \ for\ some\ k < 1$
$\Rightarrow$ 9f(n/9)$\leq kf(n)$
$\Rightarrow 9\ \times 3(n/9)^2 \leq k3n^2$
$\Rightarrow 9\ \times 3(n^2/81) \leq k3n^2$
$\Rightarrow$ 3(n^2/9) $\leq k3n^2$
$\Rightarrow$ n^2/9 $\leq kn^2$
$\Rightarrow$ k $\geq 1/9$ will satisfy the regularity condition
Case 3 applies $\Rightarrow \Theta(f(n)) \Rightarrow \Theta(n^2)$

Question 1a .2
$T(n) = 27T(n/3) + 3n^2 logn$
$Number\ of\ leaves = n^{log_b a} = n^{log_3 27} = n^{log_3 3^3} = n^{3log_3 3} = n^3$
$f(n) = 3n^2 logn$

Case 1:
$f(n) = O(n^{log_b a - \epsilon})$
$3n^2 logn = O(n^{3-\epsilon})$
$\Rightarrow$ for $\epsilon > 0, choose\ \epsilon = 0.1$
$\Rightarrow$3 n$^2 logn = O(n^{3-0.1}) \Rightarrow 3n^2 logn = O(n^{2.9})$
Case 1 applies $\Rightarrow \Theta(n^{log_b a}) \Rightarrow \Theta(n^3)$

Question 1a .3
$T(n) = 4T(n/2) + n^2/logn$

$L = Number\ of\ leaves = n^{log_b a} = n^{log_2 4} = n^{log_2 2^2} = n^{32og_2 2} = n^2$
$f(n) = n^2/logn$

Divide $f(n)/L \Rightarrow n^2/logn/n^2 = 1/logn \Rightarrow$
No Polynomial difference
cannot use case 1 nor case 3
check case 2: $f(n) \neq \Theta(L)$
$\Rightarrow$ cannot use case 2 as well
$\Rightarrow$ Master method cannot be used, no case applies

Question 1b Iterative Substitution

$T(n) = n + 2T(n/2)$
$\Rightarrow$T(n) = n+ 2(n/2+ 2T(n/4))
$\Rightarrow$T(n) = 2n+ 4T(n/4)
$\Rightarrow$T(n) = 2n+ 4(n/4+ 2T(n/8))
$\Rightarrow$T(n) = 2n+ n+ 8T(n/8)
$\Rightarrow$T(n) = 3n+ 8T(n/8)
$\Rightarrow$T(n) = 3n+ 8(n/8+ 2T(n/16))
$\Rightarrow$T(n) = 3n+ 8n/8+ 16T(n/16)
$\Rightarrow$T(n) = 4n+ 16T(n/16)

General Rule
$\Rightarrow T(n) = kn + 2^k T(n/2^k)$

Will stop when $T(1) = 1$

$\Rightarrow$T((n/2^k) = 1
$\Rightarrow$(n/2^k) = 1
$\Rightarrow$n=$2^k \Rightarrow k = logn$

Substitute the value of k
$\Rightarrow T(n) = kn + 2^k T(n/2^k)$
$\Rightarrow T(n) = logn \times n + 2^{logn} T(1)$
$\Rightarrow T(n) = nlogn + n \times 1$

$\Rightarrow T(n) = \Theta(nlogn)$

Question 2: Greedy Algorithms [21 Points]

Consider the greedy algorithms' problems we studied in class, for each pair of problem-strategy, state if it provides an optimal solution or not, and if not, give a counter example [21 points]:

1. **Fractional Knapsack** Problem with "**Select the item with the largest value**" strategy

2. **Fractional Knapsack** Problem with "**Select the item with the largest value per unit weight**" strategy

3. **0-1 Knapsack** Problem with "**Select the item with the largest value per unit weight**" strategy

4. **Activity Selection Problem** with "**Always select the activity with the minimum duration**" strategy

5. **Activity Selection Problem** with "**Always select the activity with the latest start time**" strategy

6. **Weighted Activity Selection Problem** with **"Always select the activity with the earliest finish time"** strategy

7. "**Interval Partitioning problem**" with "**Select activities with minimum conflicts first"** strategy

Question 2 Greedy Algorithms

1. Not optimal

$K = 10$

	item 1	item 2	item 3
value	10	8	7
weight	10	6	4

$Geedy = 10$
$Optimal = 15$

2. Optimal
3. Not optimal $K = 10$

	item 1	item 2
value	5	8
weight	1	10

$Geedy = 5$
$Optimal = 8$

4. Not optimal

[—— A1 ——] [—— A2 ——]
[A3]

$Geedy = 1$
$Optimal = 2$

5. optimal
6. Not optimal

[—-7—-]

[—-10—-]

$Geedy = 7$
$Optimal = 10$

7. Not optimal

[$- - A1 - -$] [— A2 —]
[————A3—-] [$A4$]

$Geedy = 3$
$Optimal = 2$

Question 3: Dynamic Programming Algorithms [20 Points]

[6 points] The array on the right resulted from solving an instance of the **0-1 Knapsack** problem using bottom-up dynamic programming. What were the <u>values</u> and <u>weights</u> of the items?

0	0	0	0
0	0	5	5
0	0	5	10

	item 1	item 2
value	5	10
weight	2	3

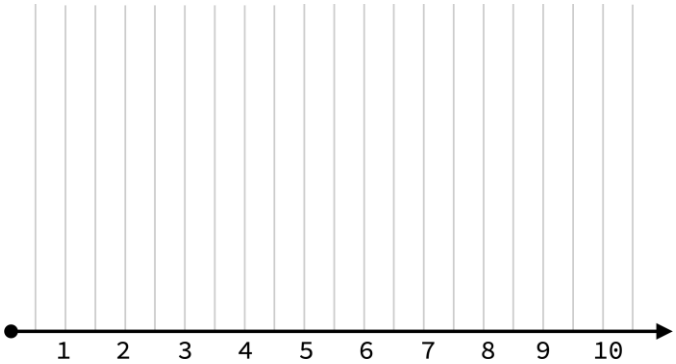
[6 points] The array on the right resulted from solving an instance of the **Longest Common Subsequence** problem using bottom-up dynamic programming. What were the two strings? (*provide any two strings that could produce this array*)

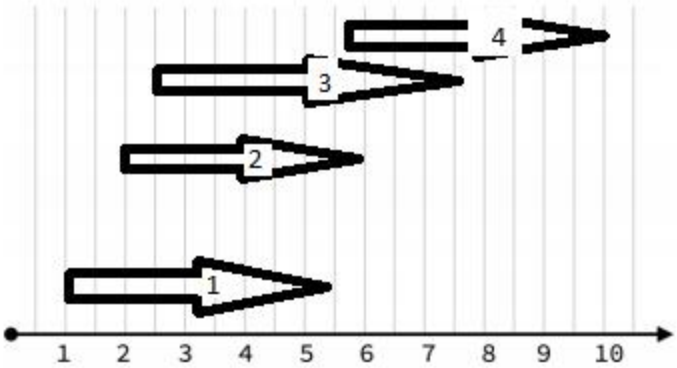
0	0	0	0
0	0	0	1
0	0	1	1
0	1	1	1

ABC and CBA

[8 points] The array on the right resulted from solving an instance of the **Weighted Activity Selection** problem using bottom-up dynamic programming. Assuming that the activity values are `v[] = {4, 3, 2, 1}`, draw on the timeline below four activities that could lead to the array on the right.

5	3	2	1	0





Question 4: Algorithms Design [29 Points]

(a) Consider the following new problem. Assume we have N items where each item has a certain weight in KG and a target location to be move to. Assume also that there is a drone that has the ability to carry items and move them from the base to their target locations. The drone can fly for a maximum distance D before requiring to recharge the battery. The drone can carry up to W kg total load in a single trip. Assume you already have the following functions:

- **TripDistance(items)**: receives a set of items and returns the total distance required to move this set of items to their destinations.
- **Distance(i):** returns the distance to move item i to its target location

I. Select a greedy strategy that attempts to maximize the number of items moved by the drone in a single trip. Assume the drone starts fully charged. Note that your greedy strategy does not have to be optimal. [8 points]

Any of the following will get full mark:

Greedy Strategy #1: Select items based on weight with minimum weight first
Greedy Strategy #2: Select items based on Distance with minimum Distance(i) first

II. Write the pseudocode of an algorithm that implements your strategy. Your algorithm should return the items selected by your strategy to be moved by the drone in a single trip. Remember that the weight of the moved items should be <= W and the TripDistance of selected times must be <= D [7 points]

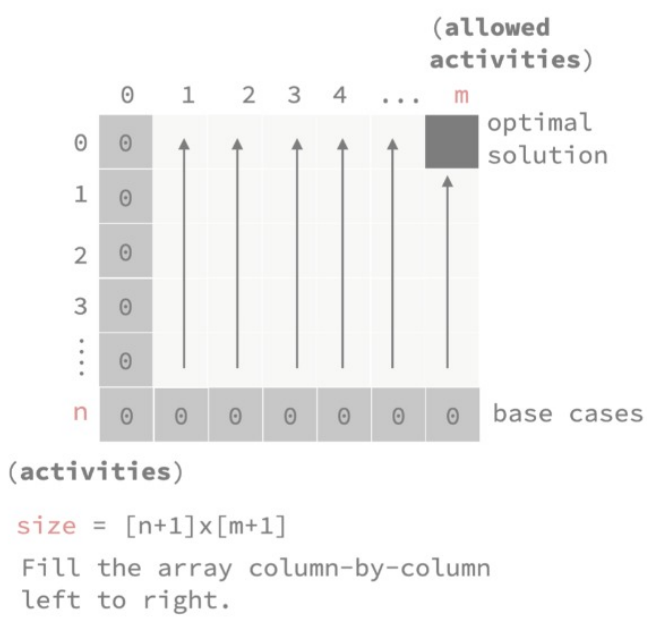
```
Sort Items based on Weight (Ascending)
 i=0
 selected_items={}
 current_weight=0
While (true)
     Get item i
    If (TripDistance(selected_items + item i) <= D ) and (i.weight + current_weight < W)
         Add Item i to selected_items
        current_weight= current_weight + i.weight

   if (i.weight > W-Current_weight)
      break;

   i=i+1
  if (i==n)
      break;
End while
Print The list of selected_items
```

D. [14 points] Consider a variant of the *weighted activity selection* problem, where taking more than m activities (out of the given n activities) is not allowed. Consider also the following recursive definition for the optimal solution **OPT**(i, m) considering the activities $i \to n-1$:

```
OPT(i, m)  =  0                                  if m <= 0 or i >= n
           =  MAX(OPT(i+1, m),
                  OPT(NEXT(i), m-1) + v[i])      otherwise
```

Convert the above recursive definition of **OPT**(i, m) to a bottom-up dynamic programming implementation. Assume the following:

- The activities are sorted according to their *start* times.
- The value of activity i is stored at v[i], which is accessible globally.
- NEXT(i) returns the index j of the first activity compatible with i, where j > i.

Note. You can implement a memoized solution instead. However, you will receive up to 6/14 points only.

```
OPT(i, m):
    CREATE result[n+1][m+1]
    Initialize result[n][] to 0
    Initialize result[][0] to 0

    FOR (i = n-1 TO 0):
        FOR (j = 1 to m):
            choice1 = result[i+1][j]
            choice2 = result[NEXT(i)][j-1] + v[i]
            result[i][j] = MAX(choice1, choice2)

    RETURN result[0][m]
```